Lab 2: Process — Answer Key

Đáp án/lời giải tham khảo cho **Lab 2: Process** (CO2018 — Hệ điều hành, GV Nguyễn Phương Duy), dựa trên nội dung `LabProcess.pdf`. Phần A giải thích các kết quả/câu hỏi "Results" của Practice 1–5 (đã có sẵn source code trong đề). Phần B là lời giải đầy đủ cho 5 bài Exercise (chỉ có khung code hoặc mô tả, chưa có lời giải).

A. Giải thích kết quả Practice (3.1–3.5)

Practice 1 — Create process (`hello_wait.c`)

Chương trình gọi `getc(stdin)` ngay sau `printf` nên process bị **block (chờ I/O)** thay vì thoát ngay — đó là lý do ta kịp `ps auxf | grep hello` thấy nó còn sống và xem được `/proc/<pid>`. Trong `/proc/<pid>/status`, trường `State` lúc này sẽ là `S (sleeping)` — đúng theo sơ đồ trạng thái Hình 2 (running → waiting vì chờ I/O).

Practice 2 — Traverse the tree of processes (`hello_fork.c`)

`fork()` tạo ra **2 tiến trình** (cha và con) cùng tiếp tục chạy từ ngay sau lệnh `fork()`, nên dòng `printf("Hello world\n")` được thực thi **2 lần độc lập** → in ra "Hello world" 2 lần. Hai PID khác nhau (3287 cha, 3288 con) xác nhận điều đó; `Ppid: 3287` trong `/proc/3288/status` và `cat /proc/3287/task/3287/children` trả về `3288` cho thấy đúng quan hệ cha–con.

Practice 3 — Examine the process memory regions (`multivar.c`)

4 địa chỉ in ra tương ứng đúng 4 vùng trong Hình 3:

- `glo_init_data` (biến toàn cục có khởi tạo) → vùng **initialized data**
- `glo_noninit_data` (biến toàn cục không khởi tạo) → vùng **uninitialized data (BSS)**
- `func` (địa chỉ hàm) → vùng **text** (mã lệnh)
- `local_data` (biến cục bộ trong `func`) → vùng **stack**

Địa chỉ `local_data` (`0x7ffe...`) lớn hơn hẳn 3 địa chỉ còn lại (`0x4000...` / `0x6000...`) — đúng với sơ đồ "low memory → high memory": text/data ở vùng thấp, stack ở vùng cao. Lệnh `cat /proc/<pid>/maps` cho thấy các vùng này được ánh xạ thành các dòng `r-xp` (text, executable), `rw-p` (data/heap), và `[stack]` (stack) tương ứng.

Practice 4.1 — Shared Memory (`reader.c` / `writer.c`)

Nếu chạy **reader trước, writer sau**, reader sẽ in `shared memory content:` **rỗng** — vì tại thời điểm reader đọc, writer **chưa kịp** `sprintf()` **ghi "hello world" vào vùng nhớ chia sẻ**. Đây chính là một **race condition** kinh điển: 2 process không có cơ chế đồng bộ nào (semaphore/mutex) để đảm bảo thứ tự ghi-trước-đọc-sau, nên kết quả phụ thuộc hoàn toàn vào ai chạy tới `sprintf` / `printf` trước.

Nếu đảo ngược thứ tự — chạy **writer trước** (cho nó kịp ghi xong rồi `sleep(10)`), rồi mới chạy **reader** trong 10 giây đó — reader sẽ đọc được đúng nội dung "hello world" vì dữ liệu đã có sẵn trong vùng nhớ chia sẻ trước khi reader truy cập. (Đây là lý do thực tế cần dùng semaphore để đồng bộ hai bên, như ở Problem 1 bên dưới.)

Practice 4.2 — Message Queue (`msgsnd.c` / `msgrcv.c`)

Message queue ở đây chỉ **một chiều**: `msgsnd` (chạy ở 1 terminal, đọc input từ bàn phím) gửi vào queue, `msgrcv` (chạy ở terminal khác) chỉ nhận. Không có chiều ngược lại (msgrcv không gửi phản hồi về msgsnd) — đúng nghĩa **HALF-DUPLEX** (bán song công — giống bộ đàm, chỉ 1 bên nói tại 1 thời điểm, không thể vừa nói vừa nghe qua lại tự do). Problem 4 yêu cầu nâng cấp lên **FULL-DUPLEX** (song công — cả 2 chiều đồng thời, giống điện thoại).

Practice 5 — Pthread (`hello_thread.c`)

`ps auxf` chỉ liệt kê **1 dòng process** `hello_thread` vì **thread không phải là process riêng** — chúng chia sẻ cùng 1 PID, cùng không gian địa chỉ (code/data/files), chỉ khác stack/register/PC riêng (đúng Hình 6). Nhưng vì có **2 thread** (`thread1`, `thread2`) cùng chạy hàm `f_count`, chương trình in ra **3 dòng thông điệp**: 1 lần "Hello world" (từ `main`), và 2 lần "Thread %s: holding %d" (1 lần mỗi thread, với giá trị `count` cuối cùng khác nhau tùy thread nào chạy xong trước — ví dụ "Thread 2: holding 10000" rồi "Thread 1: holding 20000" vì biến `count` dùng chung không được khóa, 2 thread cộng dồn lên nhau).

B. Lời giải Exercise

Problem 1 — Shared Memory: Average Movie Ratings

Đề bài: Tải file ratings (format `userID<tab>movieID<tab>rating<tab>timestamp`, 100000 dòng, 1682 phim). Viết chương trình spawn 2 tiến trình con, mỗi con đọc một phần file và tính **rating trung bình mỗi phim**, dùng **shared memory**.

Cách chia việc: thay vì tính offset byte theo kích thước file (phức tạp và dễ cắt đôi giữa dòng), mỗi con đọc **toàn bộ** file nhưng chỉ xử lý dòng có `line_index % 2 == child_id` (con 0 xử lý dòng chẵn, con 1 xử lý dòng lẻ) — đơn giản, không cần biết trước tổng số dòng, và chia tải đều.

Vì sao cần semaphore: cả 2 con cùng cộng dồn vào **chung 1 mảng** `sum[movieID]` / `count[movieID]` trong vùng nhớ chia sẻ. Nếu cả 2 cùng gặp đúng 1 movieID gần như đồng thời, thao tác `sum[id] += rating` không phải atomic (đọc → cộng → ghi) nên có thể mất mát dữ liệu (race condition, giống đúng vấn đề ở Practice 4.1). Giải pháp: dùng **1 semaphore POSIX đặt ngay trong shared memory** (`pshared=1`) làm mutex bảo vệ toàn bộ thao tác cập nhật mảng dùng chung.

```
/* avg_ratings.c — Problem 1: Shared Memory, average movie ratings */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/shm.h>
#include <sys/wait.h>
#include <semaphore.h>

#define SHM_KEY     0x2024
#define NUM_MOVIES  1682   /* movieID from 1..1682 */
#define RATINGS_FILE "u.data"  /* doi ten file neu can */

/* Du lieu nam trong shared memory, dung chung boi ca 2 con */
typedef struct {
    sem_t   mutex;                 /* bao ve sum[]/count[] */
    long    sum[NUM_MOVIES + 1];
    long    count[NUM_MOVIES + 1];
} shared_data_t;

static void process_lines(shared_data_t *data, int child_id, int
num_children) {
    FILE *fp = fopen(RATINGS_FILE, "r");
    if (!fp) { perror("fopen"); exit(1); }

    char line[256];
    long line_index = 0;
    int  userID, movieID, rating;
    long ts;

    while (fgets(line, sizeof(line), fp) != NULL) {
        /* moi con chi xu ly dong cua minh: chia tai theo line_index %
num_children */
        if (line_index % num_children == child_id) {
```

```
            if (sscanf(line, "%d\t%d\t%d\t%ld", &userID, &movieID, &rating,
&ts) == 4) {
                if (movieID >= 1 && movieID <= NUM_MOVIES) {
                    sem_wait(&data->mutex);      /* vao vung tranh chap */
                    data->sum[movieID]   += rating;
                    data->count[movieID] += 1;
                    sem_post(&data->mutex);      /* roi vung tranh chap */
                }
            }
        }
        line_index++;
    }
    fclose(fp);
}

int main(void) {
    int shmid = shmget(SHM_KEY, sizeof(shared_data_t), IPC_CREAT | 0666);
    if (shmid < 0) { perror("shmget"); exit(1); }

    shared_data_t *data = (shared_data_t *) shmat(shmid, NULL, 0);
    if (data == (void *) -1) { perror("shmat"); exit(1); }

    memset(data->sum,   0, sizeof(data->sum));
    memset(data->count, 0, sizeof(data->count));
    /* pshared = 1: semaphore duoc chia se giua CAC PROCESS (khong chi
thread) */
    sem_init(&data->mutex, 1, 1);

    pid_t pid1 = fork();
    if (pid1 == 0) {
        process_lines(data, 0, 2);   /* con 1: xu ly dong chan */
        shmdt(data);
        exit(0);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
        process_lines(data, 1, 2);   /* con 2: xu ly dong le */
        shmdt(data);
        exit(0);
    }

    /* tien trinh cha: cho 2 con xong roi moi doc ket qua */
```

```
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("movieID\tavg_rating\tnum_ratings\n");
    for (int m = 1; m <= NUM_MOVIES; m++) {
        if (data->count[m] > 0) {
            double avg = (double) data->sum[m] / data->count[m];
            printf("%d\t%.3f\t%ld\n", m, avg, data->count[m]);
        }
    }

    sem_destroy(&data->mutex);
    shmdt(data);
    shmctl(shmid, IPC_RMID, NULL);   /* xoa shared memory segment */
    return 0;
}
```

Biên dịch & chạy:

```
gcc -o avg_ratings avg_ratings.c -lpthread
./avg_ratings
```

Lưu ý:

- `fork()` được gọi **sau** `shmat()` / `sem_init()` nên cả 2 con đều thừa hưởng cùng 1 ánh xạ shared memory và cùng 1 semaphore đã khởi tạo.
- Semaphore được **đặt ngay bên trong struct của shared memory** (không phải biến cục bộ của cha) — đây là điểm hay dễ sai: nếu đặt semaphore ngoài shared memory (ví dụ `sem_init(&local_sem, 0, 1)` rồi fork), mỗi process sẽ có **bản sao riêng** của semaphore đó (vì bộ nhớ thường không được chia sẻ sau fork) nên khóa sẽ không có tác dụng giữa 2 process.
- Nếu bỏ qua semaphore (giống Practice 4.1 không đồng bộ), chương trình vẫn **thường chạy đúng** vì hai con hiếm khi cập nhật đúng cùng 1 movieID ở cùng thời điểm tuyệt đối, nhưng về lý thuyết đây là **undefined behavior** — nên giữ semaphore để kết quả luôn đúng.

Problem 2 — FIFO (Named Pipe)

Đề bài: Dùng **FIFO (named pipe)** để hiện thực cơ chế message passing, cài đặt hàm `rcv()` và `snd()` theo khung đề đã cho.

FIFO khác pipe thường (`pipe()`) ở chỗ nó có **tên trong hệ thống file** (tạo bằng `mkfifo()`), nên 2 tiến trình **không có quan hệ cha-con** vẫn mở được chung 1 FIFO theo đường dẫn, giống hệt cách `mkfifo abc_fifo_name` + `echo` / `cat` đã làm thủ công ở đầu đề.

```
/* fifo_common.h — ham dung chung cho snd.c va rcv.c */
#ifndef FIFO_COMMON_H
#define FIFO_COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <errno.h>

#define FIFO_PATH "/tmp/abc_fifo_name"
#define MSG_MAXLEN 256

/* Tao FIFO neu chua ton tai (bo qua loi "da ton tai") */
static void fifo_ensure_exists(const char *path) {
    if (mkfifo(path, 0666) == -1 && errno != EEXIST) {
        perror("mkfifo");
        exit(1);
    }
}

/* rcv: mo FIFO o che do CHI DOC, doc 1 message, tra ve so byte doc duoc */
int rcv(char *buf, size_t buf_size) {
    fifo_ensure_exists(FIFO_PATH);

    int fd = open(FIFO_PATH, O_RDONLY);   /* block cho den khi co writer mo
FIFO */
    if (fd < 0) { perror("open (rcv)"); return -1; }

    ssize_t n = read(fd, buf, buf_size - 1);
    if (n < 0) { perror("read"); close(fd); return -1; }
    buf[n] = '\0';

    close(fd);
    return (int) n;
```

```
}

/* fifo_snd: mo FIFO o che do CHI GHI, gui 1 message */
int fifo_snd(const char *msg) {
    fifo_ensure_exists(FIFO_PATH);

    int fd = open(FIFO_PATH, O_WRONLY);   /* block cho den khi co reader mo
FIFO */
    if (fd < 0) { perror("open (snd)"); return -1; }

    size_t len = strlen(msg);
    ssize_t n = write(fd, msg, len);
    if (n < 0) { perror("write"); close(fd); return -1; }

    close(fd);
    return (int) n;
}

#endif
```

```
/* fifo_reader.c — process nhan message (chay truoc o 1 terminal) */
#include "fifo_common.h"

int main(void) {
    char buf[MSG_MAXLEN];
    printf("[reader] cho message tu FIFO %s ...\n", FIFO_PATH);

    while (1) {
        int n = rcv(buf, sizeof(buf));
        if (n <= 0) continue;
        printf("[reader] nhan duoc: \"%s\" (%d bytes)\n", buf, n);
        if (strcmp(buf, "end") == 0) break;
    }
    printf("[reader] ket thuc.\n");
    return 0;
}
```

```
/* fifo_writer.c — process gui message (chay o terminal khac) */
#include "fifo_common.h"

int main(void) {
    char line[MSG_MAXLEN];
    printf("[writer] nhap tung dong, go 'end' de ket thuc:\n");

    while (fgets(line, sizeof(line), stdin) != NULL) {
        line[strcspn(line, "\n")] = '\0';   /* bo newline */
        fifo_snd(line);
        if (strcmp(line, "end") == 0) break;
    }
    printf("[writer] ket thuc.\n");
    return 0;
}
```

Biên dịch & chạy (2 terminal):

```
gcc -o fifo_reader fifo_reader.c
gcc -o fifo_writer fifo_writer.c

# Terminal 1 (chay truoc, se bi block cho writer)
./fifo_reader

# Terminal 2
./fifo_writer
```

Lưu ý: `open()` trên FIFO ở chế độ chỉ-đọc hoặc chỉ-ghi sẽ **block** cho đến khi đầu còn lại cũng mở FIFO đó — đây là cơ chế đồng bộ đơn giản đã có sẵn của FIFO, khác với shared memory (cần tự dùng semaphore như Problem 1).

Problem 3 — Serial vs Multi-thread Sum(n)

Đề bài: Tính `sum(n) = 1+2+...+n` bằng 2 cách: tuần tự (`sum_serial.c`) và chia nhỏ cho `numThreads` luồng tính song song (`sum_multi-thread.c`), có Makefile với ít nhất 2 target.

```
/* sum_serial.c */
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <n>\n", argv[0]);
        return 1;
    }
    long long n = atoll(argv[1]);
    long long sum = 0;
    for (long long i = 1; i <= n; i++) {
        sum += i;
    }
    printf("sum(%lld) = %lld\n", n, sum);
    return 0;
}
```

```
/* sum_multi-thread.c */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

typedef struct {
    long long from;   /* tinh tong tu from.. */
    long long to;      /* ..den to, bao gom ca to */
    long long partial; /* ket qua rieng cua luong nay, khong luong nao khac
dung chung */
} range_t;

void *sum_range(void *arg) {
    range_t *r = (range_t *) arg;
    long long s = 0;
    for (long long i = r->from; i <= r->to; i++) {
        s += i;
    }
    r->partial = s;
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
```

```
        fprintf(stderr, "Usage: %s <numThreads> <n>\n", argv[0]);
        return 1;
    }
    int numThreads   = atoi(argv[1]);
    long long n      = atoll(argv[2]);

    if (numThreads < 1) numThreads = 1;

    pthread_t  *tids   = malloc(sizeof(pthread_t) * numThreads);
    range_t    *ranges = malloc(sizeof(range_t)   * numThreads);

    long long chunk = n / numThreads;

    for (int t = 0; t < numThreads; t++) {
        ranges[t].from = t * chunk + 1;
        /* luong cuoi cung nhan phan du neu n khong chia het cho numThreads
*/
        ranges[t].to   = (t == numThreads - 1) ? n : (t + 1) * chunk;
        pthread_create(&tids[t], NULL, sum_range, &ranges[t]);
    }

    long long total = 0;
    for (int t = 0; t < numThreads; t++) {
        pthread_join(tids[t], NULL);
        total += ranges[t].partial;   /* moi luong ghi vao o rieng -> khong
can mutex */
    }

    printf("sum(%lld) with %d threads = %lld\n", n, numThreads, total);

    free(tids);
    free(ranges);
    return 0;
}
```

```
# Makefile
CC = gcc
CFLAGS = -O2 -Wall

all: sum_serial sum_multi_thread

sum_serial: sum_serial.c
	$(CC) $(CFLAGS) -o sum_serial sum_serial.c

sum_multi_thread: sum_multi-thread.c
	$(CC) $(CFLAGS) -pthread -o sum_multi_thread sum_multi-thread.c

clean:
	rm -f sum_serial sum_multi_thread

.PHONY: all clean
```

Biên dịch & chạy:

```
make
./sum_serial 1000000
./sum_multi_thread 10 1000000
```

Tại sao không cần mutex: mỗi luồng ghi kết quả riêng vào `ranges[t].partial` (ô nhớ **riêng biệt**, không chung với luồng khác) — không có 2 luồng nào cùng ghi vào 1 biến, nên không có race condition, không cần đồng bộ. Tổng cuối cùng (`total += ...`) cũng an toàn vì chỉ thực thi tuần tự ở **main thread sau khi** `pthread_join()` đã đảm bảo luồng đó chạy xong.

Về speed-up: với `n` đủ lớn (ví dụ ≥ 10 triệu) và số luồng ≤ số nhân CPU thực tế, bản multi-thread thường nhanh hơn rõ rệt vì công việc thuần tính toán (CPU-bound) được chia đều cho nhiều lõi chạy song song. Với `n` nhỏ hoặc số luồng quá nhiều (vượt số nhân CPU), chi phí tạo/đồng bộ thread (`pthread_create` / `pthread_join`) có thể làm bản multi-thread **chậm hơn** bản serial — nên đo thử bằng `time ./sum_multi_thread <t> <n>` với nhiều giá trị `t` khác nhau để thấy rõ.

Problem 4 — Two-way Message Queue

Đề bài: Practice 3.4.2 chỉ là HALF-DUPLEX (1 chiều: msgsnd → msgrcv). Bài này yêu cầu nâng cấp thành **two-way (full-duplex)** bằng cơ chế multi-thread.

Thiết kế: dùng **2 message type khác nhau trên cùng 1 message queue** — `mtype=1` cho chiều A→B, `mtype=2` cho chiều B→A. Mỗi bên (peer) chạy đồng thời **2 thread**: 1 thread gửi (đọc từ bàn phím rồi `msgsnd`) và 1 thread nhận (`msgrcv` liên tục, luôn lắng nghe) — nhờ vậy cả 2 bên có thể **vừa gửi vừa nhận** cùng lúc, không phải chờ lượt như bản gốc.

```
/* duplex_peer.c — Problem 4: full-duplex qua 1 message queue, 2 thread/peer
 * Bien dich 2 lan thanh 2 chuong trinh khac nhau (hoac dung tham so):
 *   ./peer A   -> gui voi mtype=1, nhan mtype=2
 *   ./peer B   -> gui voi mtype=2, nhan mtype=1
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

#define MSG_KEY   0x2024
#define PERMS     0666

struct my_msgbuf {
    long mtype;
    char mtext[256];
};

static int   msqid;
static long  send_type, recv_type;
static const char *my_name;

/* Thread gui: doc tu ban phim, msgsnd voi send_type */
void *sender_thread(void *arg) {
    struct my_msgbuf buf;
    char line[256];

    while (fgets(line, sizeof(line), stdin) != NULL) {
        line[strcspn(line, "\n")] = '\0';
        buf.mtype = send_type;
        strncpy(buf.mtext, line, sizeof(buf.mtext) - 1);
        buf.mtext[sizeof(buf.mtext) - 1] = '\0';

        if (msgsnd(msqid, &buf, strlen(buf.mtext) + 1, 0) == -1) {
```

```
            perror("msgsnd");
        }
        if (strcmp(line, "end") == 0) break;
    }
    return NULL;
}

/* Thread nhan: msgrcv lien tuc tren recv_type, khong block thread gui */
void *receiver_thread(void *arg) {
    struct my_msgbuf buf;
    while (1) {
        if (msgrcv(msqid, &buf, sizeof(buf.mtext), recv_type, 0) == -1) {
            perror("msgrcv");
            continue;
        }
        printf("\n[%s nhan duoc] %s\n> ", my_name, buf.mtext);
        fflush(stdout);
        if (strcmp(buf.mtext, "end") == 0) break;
    }
    return NULL;
}

int main(int argc, char *argv[]) {
    if (argc != 2 || (argv[1][0] != 'A' && argv[1][0] != 'B')) {
        fprintf(stderr, "Usage: %s <A|B>\n", argv[0]);
        return 1;
    }
    my_name = argv[1];

    if (my_name[0] == 'A') { send_type = 1; recv_type = 2; }
    else                   { send_type = 2; recv_type = 1; }

    if ((msqid = msgget(MSG_KEY, PERMS | IPC_CREAT)) == -1) {
        perror("msgget");
        exit(1);
    }

    pthread_t t_send, t_recv;
    pthread_create(&t_recv, NULL, receiver_thread, NULL);
    pthread_create(&t_send, NULL, sender_thread,   NULL);

    pthread_join(t_send, NULL);
    pthread_join(t_recv, NULL);
```

```
    /* Chi 1 trong 2 peer (vd: A) nen xoa queue sau khi ca 2 da thoat */
    /* msgctl(msqid, IPC_RMID, NULL); */
    return 0;
}
```

Biên dịch & chạy (2 terminal):

```
gcc -pthread -o peer duplex_peer.c

# Terminal 1
./peer A

# Terminal 2
./peer B
```

Gõ dòng ở terminal nào, terminal kia **lập tức** nhận được (không cần đợi terminal kia gửi trước) — khác hẳn Practice 3.4.2 chỉ đi được 1 chiều.

Vì sao cần 2 thread riêng: `msgrcv()` là lời gọi **block** (chờ đến khi có message đúng `mtype`) — nếu gọi nó trên cùng 1 luồng với việc đọc bàn phím, chương trình sẽ "kẹt" ở 1 trong 2 việc tại 1 thời điểm. Tách ra 2 thread độc lập (giống Practice 3.5) cho phép cả gửi lẫn nhận cùng chờ độc lập mà không chặn lẫn nhau.

Problem 5 — mmap File Sharing

Đề bài: Dùng `mmap()` để ánh xạ 1 file vào không gian địa chỉ của 2 tiến trình, dùng nó để chia sẻ dữ liệu giữa 2 bên — cùng mục đích với Practice 4.1 (shared memory) nhưng dùng cơ chế **ánh xạ file** (`MAP_SHARED`) thay vì `shmget`/`shmat`.

Ý tưởng: tạo/mở 1 file chung (ví dụ `shared.dat`), `ftruncate()` cho nó đủ lớn, rồi cả writer và reader cùng `mmap()` file đó với `MAP_SHARED` — khi writer ghi vào vùng nhớ đã mmap, thay đổi được **cả kernel đồng bộ ngược xuống file** và cũng **thấy được ngay** từ vùng nhớ đã mmap của reader (không cần gọi `read()`/`write()` lại qua file).

```
/* mmap_writer.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
```

```
#include <sys/stat.h>

#define SHM_FILE "shared.dat"
#define SHM_SIZE 4096

int main(void) {
    int fd = open(SHM_FILE, O_RDWR | O_CREAT, 0666);
    if (fd < 0) { perror("open"); exit(1); }

    if (ftruncate(fd, SHM_SIZE) == -1) {   /* dat kich thuoc file truoc khi
mmap */
        perror("ftruncate");
        exit(1);
    }

    char *map = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); exit(1); }

    close(fd);   /* sau khi mmap thanh cong, co the dong fd, mapping van con
hieu luc */

    printf("[writer] PID = %d, mapped at %p\n", getpid(), (void *) map);
    snprintf(map, SHM_SIZE, "hello world (tu writer, pid=%d)", getpid());
    printf("[writer] da ghi: \"%s\"\n", map);

    printf("[writer] ngu 10s de reader kip doc...\n");
    sleep(10);

    munmap(map, SHM_SIZE);
    return 0;
}
```

```
/* mmap_reader.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>

#define SHM_FILE "shared.dat"
#define SHM_SIZE 4096
```

```
int main(void) {
    int fd = open(SHM_FILE, O_RDWR);   /* file phai da duoc writer tao truoc
*/
    if (fd < 0) { perror("open (hay chay writer truoc!)"); exit(1); }

    char *map = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                     MAP_SHARED, fd, 0);
    if (map == MAP_FAILED) { perror("mmap"); exit(1); }

    close(fd);

    printf("[reader] PID = %d, mapped at %p\n", getpid(), (void *) map);
    printf("[reader] doc duoc: \"%s\"\n", map);

    munmap(map, SHM_SIZE);
    return 0;
}
```

Biên dịch & chạy (2 terminal, giống Practice 4.1):

```
gcc -o mmap_writer mmap_writer.c
gcc -o mmap_reader mmap_reader.c

# Terminal 1: chay truoc, se ngu 10s sau khi ghi
./mmap_writer

# Terminal 2: chay trong luc writer dang ngu
./mmap_reader
```

So sánh với shared memory (Practice 4.1):

- `shmget` / `shmat` tạo 1 vùng nhớ ảo, không gắn với file cụ thể nào trên đĩa; dữ liệu **mất khi xóa segment** (`shmctl(..., IPC_RMID, ...)`).
- `mmap()` với 1 file thật (không phải `MAP_ANONYMOUS`) **đồng bộ nội dung xuống đĩa** — dữ liệu vẫn còn sau khi cả 2 process kết thúc (có thể `cat shared.dat` để kiểm tra), và process thứ 3 mở file sau đó vẫn đọc được nội dung đã ghi.
- Giống hệt Practice 4.1, đây vẫn là **race condition** nếu không đảm bảo writer ghi trước reader đọc (demo trên dùng `sleep(10)` cho đơn giản; thực tế nên dùng semaphore đặt trong vùng mmap, tương tự Problem 1).